

Name: - Shubham Das

S. A. P I. D: - 590033500

Date: - 23/09/2026

Programme: - B. TECH (CSE)

Assignment: - Write a C program to perform the following menu-driven operations on given array:

1. Sort the elements using the Bubble Sort algorithm and perform a Binary search.
2. Perform Insertion and Deletion in a given valid position.
3. Rotate the array elements to the right by k positions.
4. Exit

Part 1: Thorough understanding of the problem statement

Input: -

1. *n* (Active number of elements in the array, Variable: int, Unit: unitless integer)
2. *Arr[100]* (1D array capacity, Variable: int, Unit: scalar values)
3. *choice* (Menu option selected, Variable: int, Unit: unitless)
4. *key* (Target element for Binary Search, Variable: int)
5. *pos, val* (Index position and value for insertion/deletion, Variables: int)
6. *k* (Number of positions to rotate right, Variable: int)

Output: -

1. Sorted array and search index (Variable: int)
2. Modified array after insertion/deletion (Variables: int array elements)
3. Shifted array after right rotation (Variables: int array elements)

Formula: -

1. Bubble Sort: $\text{if } (arr[j] > arr[j+1]) \{temp = arr[j]; arr[j] = arr[j+1]; arr[j+1] = temp;\}$
2. Binary Search: $mid = left + (right - left) / 2;$
3. Insertion Shift: $\text{for } (i = n; i > pos; i--) arr[i] = arr[i-1];$
4. Deletion Shift: $\text{for } (i = pos; i < n - 1; i++) arr[i] = arr[i+1];$
5. Right Rotate (Reversal): Reverse whole array, reverse first *k*, reverse remaining *n-k*.

Part 2: - Writing, compiling and evaluating the algorithm

Step 1: - Start.

Start 2: - Declare a fixed-size integer array *arr[100]* and variable *n* to track the active element count. Read the initial array elements from the user.

Step 3: - Initiate a *do-while* loop for the menu.

Step 4 (Case 1- Sort & Search): - Apply Bubble Sort using nested loops. Display the sorted array. Prompt the user for a search *key*. Apply Binary

Search logic. If $arr[mid] == key$, print the index; else, adjust *left* or *right* bounds. If the loop ends, print "Not found".

Step 5 (Case 2- Insert & Delete): - Prompt for *val* and *pos* (0 to *n*). Validate *pos*. Shift elements from *n* down to *pos* one step to the right. Set

arr[pos] = val and increment *n*. Print array.

Prompt for deletion *pos* (0 to *n*-1). Validate *pos*.

Shift elements from *pos* to *n*-1 one step to the left. Decrement *n*.

Print array.

Step 6 (Case 3- Right Rotate): - Prompt for *k*. Calculate effective rotations: $k = k \% n$.

Use an in-place array reversal algorithm to flip the entire array,

Then flip the first *k* elements, and finally flip the remaining *n*-*k* elements. Print the rotated array.

Step 7 (Case 4- Exit): - Terminate the loop.

Step 8: - Stop.

Part 3: - Writing and compiling the C program

GNU nano 9.0

```
#include <stdio.h>

// Helper function to display the array
void display(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

// Helper function to reverse a section of the array
void reverse(int arr[], int start, int end) {
    while (start < end) {
        int temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;
        start++;
        end--;
    }
}

int main() {
    int arr[100];
    int n, choice;
    int key, pos, val, k;
    int temp, left, right, mid, found;

    printf("Enter the number of elements in the array (max 100): ");
    scanf("%d", &n);

    if (n <= 0 || n > 100) {
        printf("Invalid array size. Exiting...\n");
        return 1;
    }

    printf("Enter the %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    do {
        printf("\n===== ARRAY OPERATIONS MENU =====\n");
        printf("1. Sort (Bubble) & Search (Binary)\n");
        printf("2. Insert & Delete at a valid position\n");
        printf("3. Rotate the array elements to the right by k\n");
        printf("4. Exit\n");
        printf("Enter your choice (1-4): ");
        scanf("%d", &choice);
```

GNU nano 9.0

```
switch (choice) {
    case 1:
        // Bubble Sort
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
        printf("Array after Bubble Sort: ");
        display(arr, n);

        // Binary Search
        printf("Enter element to search using Binary Search: ");
        scanf("%d", &key);
        left = 0; right = n - 1; found = -1;
        while (left <= right) {
            mid = left + (right - left) / 2;
            if (arr[mid] == key) {
                found = mid;
                break;
            }
            if (arr[mid] < key) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        if (found != -1) {
            printf("Element %d found at index %d (0-based).\n", key, found);
        } else {
            printf("Element %d not found in the array.\n", key);
        }
        break;

    case 2:
        // Insertion
        printf("\n--- Insertion ---\n");
        printf("Enter value to insert: ");
        scanf("%d", &val);
        printf("Enter index position to insert (0 to %d): ", n);
        scanf("%d", &pos);
        if (pos >= 0 && pos <= n && n < 100) {
            for (int i = n; i > pos; i--) {
                arr[i] = arr[i - 1];
```

```

    }
    arr[pos] = val;
    n++;
    printf("Array after insertion: ");
    display(arr, n);
} else {
    printf("Invalid insertion position or array is full.\n");
}

// Deletion
printf("\n--- Deletion ---\n");
printf("Enter index position to delete (0 to %d): ", n - 1);
scanf("%d", &pos);
if (pos >= 0 && pos < n) {
    for (int i = pos; i < n - 1; i++) {
        arr[i] = arr[i + 1];
    }
    n--;
    printf("Array after deletion: ");
    display(arr, n);
} else {
    printf("Invalid deletion position.\n");
}
break;

case 3:
    // Right Rotate by k
    printf("Enter number of positions to right rotate (k): ");
    scanf("%d", &k);
    if (n > 0) {
        k = k % n;
        if (k < 0) k += n; // Handle negative inputs mathematically

        // Reversal algorithm for right rotation
        reverse(arr, 0, n - 1); // Reverse whole array
        reverse(arr, 0, k - 1); // Reverse first k elements
        reverse(arr, k, n - 1); // Reverse remaining n-k elements

        printf("Array after rotating right by %d positions: ", k);
        display(arr, n);
    }
    break;

case 4:
    printf("Exiting program. Goodbye!\n");
    break;

default:

```

```
GNU nano 9.0
}

// Deletion
printf("\n--- Deletion ---\n");
printf("Enter index position to delete (0 to %d): ", n - 1);
scanf("%d", &pos);
if (pos >= 0 && pos < n) {
    for (int i = pos; i < n - 1; i++) {
        arr[i] = arr[i + 1];
    }
    n--;
    printf("Array after deletion: ");
    display(arr, n);
} else {
    printf("Invalid deletion position.\n");
}
break;

case 3:
    // Right Rotate by k
    printf("Enter number of positions to right rotate (k): ");
    scanf("%d", &k);
    if (n > 0) {
        k = k % n;
        if (k < 0) k += n; // Handle negative inputs mathematically

        // Reversal algorithm for right rotation
        reverse(arr, 0, n - 1); // Reverse whole array
        reverse(arr, 0, k - 1); // Reverse first k elements
        reverse(arr, k, n - 1); // Reverse remaining n-k elements

        printf("Array after rotating right by %d positions: ", k);
        display(arr, n);
    }
    break;

case 4:
    printf("Exiting program. Goodbye!\n");
    break;

default:
    printf("Error: Invalid choice. Try again.\n");
}
} while (choice != 4);

return 0;
}
```

Part 4: - Running and executing the algorithm on a platform (Kali Linux VM)

- Step 1: - Open the terminal
- Step 2: - Create the source code
- Step 3: - Write and Save Code
- Step 4: - Compile the program

Step 6: - Run the Binary Executable Program

```
(kali㉿kali) -[~]  
$ nano advanced_array.c  
  
(kali㉿kali) -[~]  
$ gcc advanced_array.c -o advanced_array
```

```

(kali㉿kali)-[~]
└─$ ./advanced_array
Enter the number of elements in the array (max 100): 5
Enter the 5 elements:
5 1 4 2 8

===== ARRAY OPERATIONS MENU =====
1. Sort (Bubble) & Search (Binary)
2. Insert & Delete at a valid position
3. Rotate the array elements to the right by k
4. Exit
Enter your choice (1-4): 1
Array after Bubble Sort: 1 2 4 5 8
Enter element to search using Binary Search: 4
Element 4 found at index 2 (0-based).

===== ARRAY OPERATIONS MENU =====
1. Sort (Bubble) & Search (Binary)
2. Insert & Delete at a valid position
3. Rotate the array elements to the right by k
4. Exit
Enter your choice (1-4): 1
Array after Bubble Sort: 1 2 4 5 8
Enter element to search using Binary Search: 9
Element 9 not found in the array.

===== ARRAY OPERATIONS MENU =====
1. Sort (Bubble) & Search (Binary)
2. Insert & Delete at a valid position
3. Rotate the array elements to the right by k
4. Exit
Enter your choice (1-4): 2

--- Insertion ---
Enter value to insert: 9
Enter index position to insert (0 to 5): 0
Array after insertion: 9 1 2 4 5 8

--- Deletion ---
Enter index position to delete (0 to 5): 3
Array after deletion: 9 1 2 5 8

===== ARRAY OPERATIONS MENU =====
1. Sort (Bubble) & Search (Binary)
2. Insert & Delete at a valid position
3. Rotate the array elements to the right by k
4. Exit
Enter your choice (1-4): 3
Enter number of positions to right rotate (k): 1
Array after rotating right by 1 positions: 8 9 1 2 5

===== ARRAY OPERATIONS MENU =====

```

```

3. Rotate the array elements to the right by k
4. Exit
Enter your choice (1-4): 2

--- Insertion ---
Enter value to insert: 9
Enter index position to insert (0 to 5): 0
Array after insertion: 9 1 2 4 5 8

--- Deletion ---
Enter index position to delete (0 to 5): 3
Array after deletion: 9 1 2 5 8

===== ARRAY OPERATIONS MENU =====
1. Sort (Bubble) & Search (Binary)
2. Insert & Delete at a valid position
3. Rotate the array elements to the right by k
4. Exit
Enter your choice (1-4): 3
Enter number of positions to right rotate (k): 1
Array after rotating right by 1 positions: 8 9 1 2 5

===== ARRAY OPERATIONS MENU =====
1. Sort (Bubble) & Search (Binary)
2. Insert & Delete at a valid position
3. Rotate the array elements to the right by k
4. Exit
Enter your choice (1-4): 2

--- Insertion ---
Enter value to insert: 99
Enter index position to insert (0 to 5): 1
Array after insertion: 8 99 9 1 2 5

--- Deletion ---
Enter index position to delete (0 to 5): 3
Array after deletion: 8 99 9 2 5

===== ARRAY OPERATIONS MENU =====
1. Sort (Bubble) & Search (Binary)
2. Insert & Delete at a valid position
3. Rotate the array elements to the right by k
4. Exit
Enter your choice (1-4): 2

--- Insertion ---
Enter value to insert: 5
Enter index position to insert (0 to 5): 99
Invalid insertion position or array is full.

--- Deletion ---
Enter index position to delete (0 to 4): █

```

Part 5: - Standard Test Table

TEST CASE	SCENARIO/OPERATION	INPUT	EXPECTED OUTPUT	RESULT
1	Sort & Search	Array: 5 1 4 -> Choice 1 -> Search: 4	Sorted: 1 4 5 Found at index 1.	PASS
2	Search Miss	Array: 1 4 5 -> Choice 1 -> Search: 9	Element 9 not found.	PASS
3	Insert Element	Choice 2 -> Val: 9, Pos: 0	Array: 9 1 4 5	PASS
4	Delete Element	Following TC3 -> Pos: 3	Array: 9 1 4	PASS
5	Rotate Right	Choice 3 -> k = 1 on array 9 1 4	Rotated: 4 9 1	PASS
6	Invalid Bounds	Choice 2 -> Insert Pos: 99	"Invalid insertion position..."	PASS

Part 6: - Evaluation

A static array spanning 100 integer positions is deliberately utilized to emulate dynamic capabilities in standard C without explicitly allocating blocks natively in heap memory using *malloc*. This approach simplifies index checking bounds, averts dynamic resizing complexity, and preserves fast, localized stack memory lookups.